



kubernetes



The Twlve-Factor APP

Nell'era moderna, il software viene fornito sempre più di frequente come servizio (delivered as a service).

Si parla di web app o software as a service (SaaS).

La realizzazione di Software as a Service (SaaS) è uno scenario sempre più frequente, pertanto nasce sempre più spesso la domanda : quale metodologia utilizzare per lo sviluppo?

Una delle risposte è lo sviluppo Agile con framework come Scrum e XP.

Esiste però un nuovo punto di vista per sviluppatori e DevOps chiamata "twelve-factor app".

La twelve-factor app è una metodologia di sviluppo orientata alla costruzione di applicazioni software-as-a-service che:

- **Seguono un formato dichiarativo per l'automazione della configurazione, minimizzando tempi e costi di ingresso per ogni sviluppatore che si aggiunge al progetto;**
- **Si interfacciano in modo pulito con il sistema operativo sottostante, in modo tale da offrire la massima portabilità sui vari ambienti di esecuzione;**
- **Sono adatte allo sviluppo sulle più recenti cloud platform, ovviando alla necessità di server e amministrazioni di sistema;**
- **Minimizzano la divergenza tra sviluppo e produzione, permettendo il continuous deployment per una massima "agilità";**
- **Possono scalare significativamente senza troppi cambiamenti ai tool, all'architettura e al processo di sviluppo;**

I dodici fattori o principi su cui basare lo sviluppo sono i seguenti:

- **Codebase**
- **Dipendenze**
- **Configurazione**
- **Backing Service**
- **Build, release, esecuzione**
- **Processi**
- **Binding delle Porte**
- **Concorrenza**
- **Rilasciabilità**
- **Parità tra Sviluppo e Produzione**
- **Log**
- **Processi di Amministrazione**

12 Factors



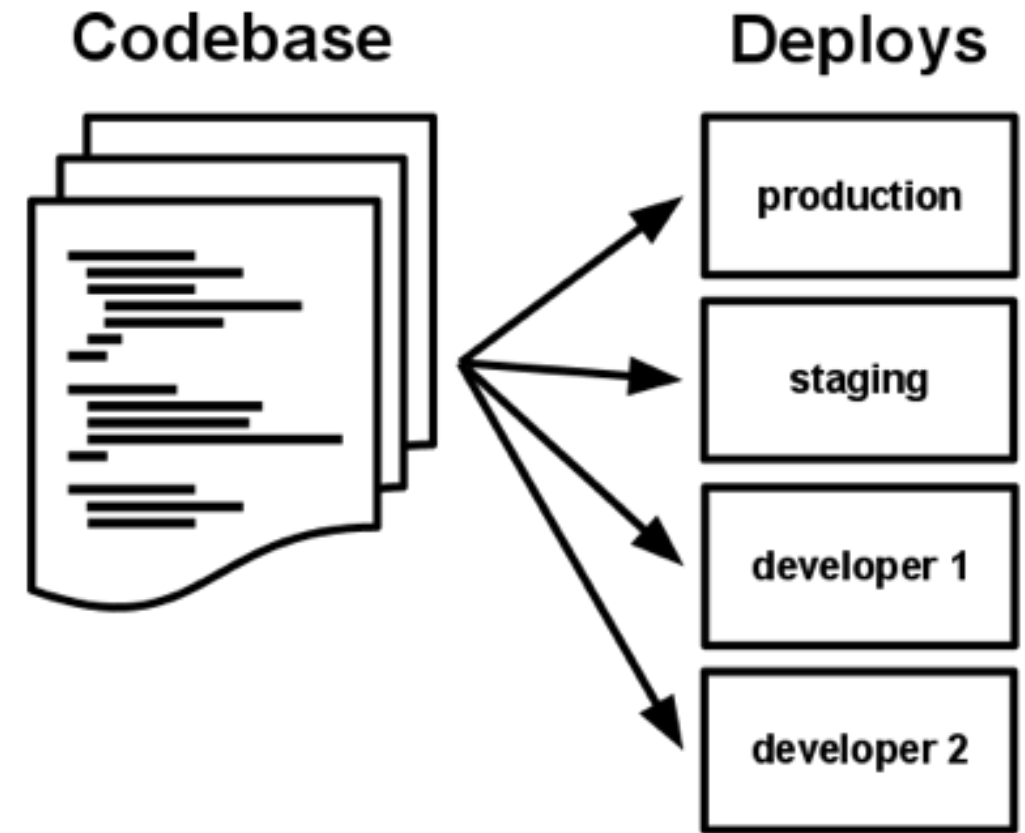


Codebase

Una sola codebase sotto controllo di versione, tanti deploy-

Un'app conforme alla metodologia twelve-factor è sempre sotto un sistema di controllo di versione, sia essa Git, Mercurial, o Subversion. Una copia della codebase è detta code repository, oppure più in breve code repo o repo.

Una codebase è quindi un singolo repository (in un sistema centralizzato come Subversion), oppure un set di repo che condividono una root commit (in un sistema di controllo decentralizzato come Git).



C'è sempre una relazione uno-ad-uno tra codebase e applicazione:

- **Se ci sono più codebase, non si parla più di applicazione ma di sistema distribuito. Ogni componente in un sistema distribuito è un'applicazione, e ognuna di queste applicazioni può individualmente aderire alla metodologia twelve-factor.**
- **Se più applicazioni condividono lo stesso codice si ha una violazione del twelve-factor. La soluzione è, ovviamente, quella di sistemare il codice in modo adeguato, in modo tale da essere incluso eventualmente dove necessario tramite un dependency manager.**

Quindi: una sola codebase per applicazione, ma ci saranno comunque tanti deployment della stessa app.

Per deploy si intende un'istanza dell'applicazione. Può essere il software in produzione, oppure una delle varie istanze in staging. Ancora, un deploy può essere la copia posseduta dal singolo sviluppatore nel suo ambiente locale.

La codebase rimane comunque sempre la stessa su tutti i deployment, anche se potrebbero essere attive diverse versioni nello stesso istante. Si pensi per esempio a uno sviluppatore che possiede dei commit in più che non ha ancora mandato in staging.

Nonostante questo, comunque, rimane la condivisione della stessa codebase, nonostante la possibilità di avere più deploy della stessa app.



Dipendenze

Dipendenze dichiarate e isolate.

Molti linguaggi di programmazione offrono dei sistemi di packaging per la distribuzione delle proprie librerie, come CPAN per Perl o Rubygems per Ruby.

Le librerie installate attraverso questi sistemi, inoltre, possono essere identificate come “system-wide” (attive a livello di sistema), oppure posizionate nella directory della singola applicazione (in questo caso si parla di “vendoring” o “bundling”).

Un'applicazione che aderisce alla twelve-factor non si basa mai sull'esistenza implicita di librerie system-wide.

Le dipendenze vengono tutte dichiarate, tramite un manifest dedicato. Inoltre, viene contemplato anche l'uso di un tool di isolamento delle dipendenze durante l'esecuzione, in modo tale da assicurarsi che non ci siano delle "dipendenze implicite" che creino interferenze nel sistema in cui ci si trova.

La specifica completa ed esplicita delle dipendenze si applica in modo uniforme: sia in production che in sviluppo.

Per esempio, Bundler per Ruby offre il supporto di un file-manifesto Gemfile da usare per la dichiarazione delle dipendenze e bundle exec per il loro isolamento.

In Python invece troviamo altri due tool per questi scopi – Pip viene usato per la dichiarazione e Virtualenv per l'isolamento.

Anche C ha Autoconf per la dichiarazione di dipendenze, mentre lo static linking si occupa dell'isolamento.

Non importa quale sia il toolchain usato, le operazioni di dichiarazione e isolamento vanno sempre effettuate.

In caso contrario, l'applicazione non aderisce più alla metodologia.

Un altro importante beneficio di una dichiarazione esplicita delle dipendenze sta nel fatto che semplifica di molto la configurazione iniziale per gli sviluppatori appena entrati a lavorare al progetto.

Il nuovo arrivato non dovrà fare altro che effettuare un check out della codebase nella propria macchina di sviluppo, occupandosi di dover installare solo ed esclusivamente le dipendenze, appunto, dichiarate.

Molto spesso è inoltre presente un build command che permette di automatizzare il processo. Per Ruby/Bundler si usa bundle install, mentre per Clojure/Leiningen c'è lein deps.

Ogni applicazione che aderisce alla metodologia twelve-factor, inoltre, non si basa mai sull'esistenza di un qualsiasi tool di sistema.

Alcuni esempi sono ImageMagick o curl.

Nonostante questi software esistano già su buona parte dei sistemi in circolazione, non è comunque detto che siano presenti su tutti quelli su cui girerà l'applicazione in futuro.

Se l'app non può fare a meno di questo tool, si dovrebbe prendere in considerazione l'idea di "vendorizzarlo" nell'applicazione stessa.

Configurazione

The background is a deep blue with a complex pattern of concentric circles and radial lines, creating a sense of depth and movement, similar to a tunnel or a data visualization. The lines are lighter blue and white, and the overall effect is futuristic and technological.

Memorizza le informazioni di configurazione nell'ambiente.

La “configurazione” di un'applicazione è tutto quello che può variare da un deployment all'altro (staging, production, ambienti di sviluppo). Ad esempio:

- **Valori da usare per connettersi a un database, Memcached, oppure altri backing service;**
- **Credenziali per servizi esterni, come Amazon S3 o Twitter;**
- **Valori eventualmente definiti per i singoli deployment, come l'hostname;**

Molto spesso, queste informazioni vengono memorizzate come costanti nel codice: la cosa viola la metodologia twelve-factor, che richiede una separazione ben definita delle impostazioni di configurazione dal codice. Le impostazioni possono infatti variare da un deployment all'altro: il codice, invece, no.

Il codice dell'applicazione, infatti, potrebbe essere reso open-source in ogni momento: un buon motivo per separare le due cose.

Nota che comunque la definizione di “configurazione” non include eventuali configurazione interne dell'applicazione, come config/routes.rb in Rails, o come i moduli di codice sono connessi in Spring.

Questo tipo di configurazione non varia da deployment a deployment: come giusto che sia, quindi, rimane nel codice.

Un ottimo approccio al “rispetto” di questo principio consiste nell’usare dei file di configurazione non coinvolti dal version control, come per esempio config/database.yml in Rails.

Stiamo parlando di un miglioramento enorme rispetto all’uso di costanti nel codice, ma c’è da dire la cosa ha il suo lato negativo: basta poco per sbagliarsi e includere nel repo il file di configurazione che, invece, non dovrebbe essere lì.

C’è una certa tendenza, infatti, a non avere tutti i file di configurazione necessari nello stesso posto. Inoltre, i vari formati tendono a essere collegati a un certo linguaggio/framework.

L'applicazione che rispetta la metodologia twelve-factor memorizza tutte le impostazioni in variabili d'ambiente (spesso dette env vars o env).

Le variabili d'ambiente sono molto semplici da cambiare di deployment in deployment senza dover toccare il codice direttamente.

Inoltre, a differenza dei file di configurazione classici, c'è una probabilità molto bassa di venire inclusi nel repo.

Infine, questi file sono indipendenti sia dal linguaggio che dal sistema operativo utilizzato.

Un altro aspetto del config management è il raggruppamento.

A volte, infatti, alcune applicazioni prevedono la memorizzazione delle impostazioni in gruppi (chiamati spesso “ambienti”) dal nome ben preciso: “development”, “test” e “production”, per esempio.

Questo metodo non scala correttamente, se ci pensi: potrebbero essere necessari nuovi ambienti, come “staging” e “qa”.

Oppure, i vari sviluppatori potrebbero aver bisogno di creare i propri ambienti “speciali”, come “joes-staging” e così via.

Il risultato? Una quantità di combinazioni ingestibile e disordinata.

In una buona twelve-factor app, le variabili di ambiente sono controllate in modo più “granulare”, in modo totalmente ortogonale alle altre.

Non sono mai raggruppate e classificate sotto “ambienti” specifici, ma vengono gestite in modo totalmente indipendente per ogni deployment.

Il prodotto finale ne risente positivamente in termini di scalabilità.



Backing Service

Tratta i backing service come “risorse”.

Un “backing service” è un qualsiasi servizio che l’applicazione consuma attraverso la rete durante la sua esecuzione.

Alcuni esempi includono i database (come MySQL o CouchDB), servizi di messaging/code (come RabbitMQ oppure Beanstalkd), servizi SMTP per la posta (come Postfix) e sistemi di cache (come Memcached).

Un backing service (prendiamo ad esempio un database) è tradizionalmente gestito dallo stesso amministratore di sistema, al deployment dell'applicazione.

In aggiunta a questi servizi gestiti in locale potrebbero esserne presenti altri, forniti da terze parti.

Parliamo di servizi SMTP (come Postmark), servizi di raccolta metriche (come New Relic o Loggly), servizi per asset (come Amazon S3), e anche servizi accessibili via API (come Twitter, Google Maps, o Last.fm).

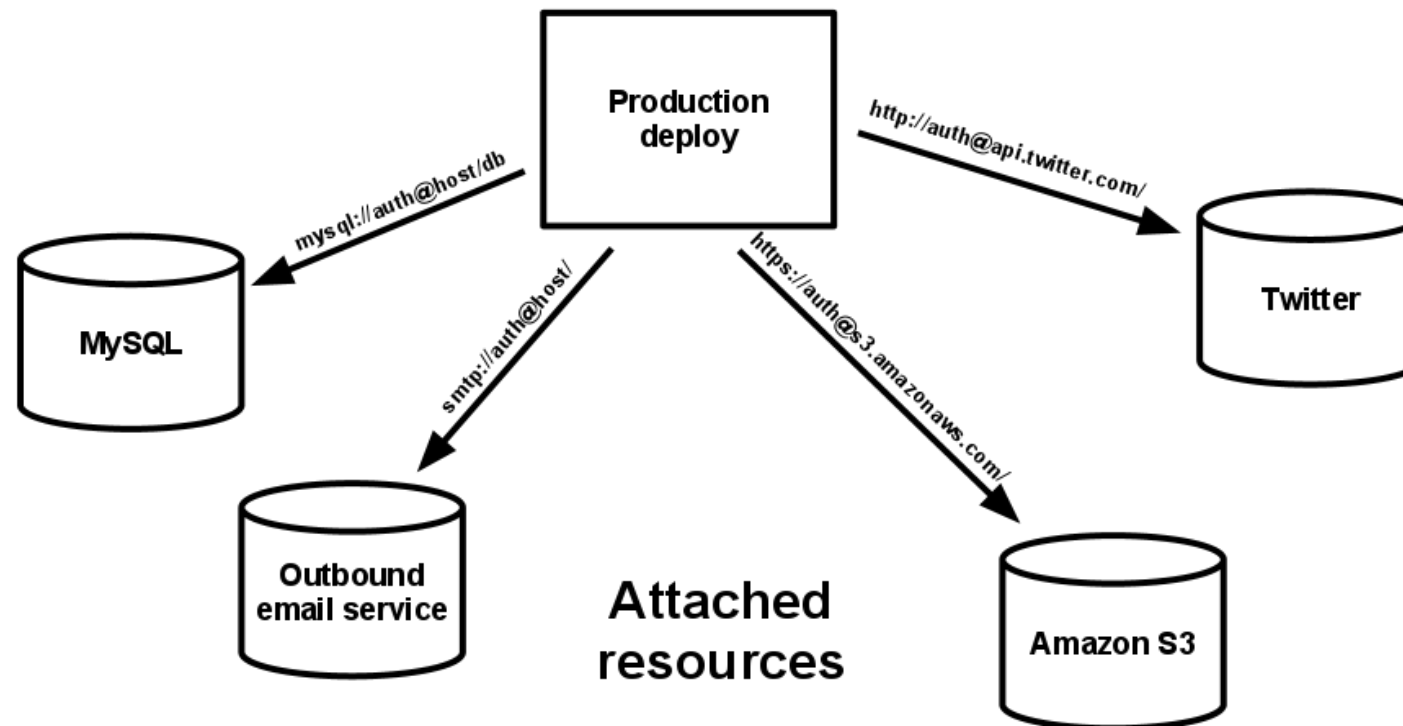
Il codice di un'app twelve-factor non fa distinzioni tra servizi in locale o third party.

Per l'applicazione, entrambi sono risorse connesse, accessibili via URL (o tramite un altro locator) e credenziali memorizzate nell'opportuno file di configurazione.

A un qualsiasi deployment di un'applicazione twelve-factor si deve poter permettere di passare velocemente da un database MySQL locale a uno third party (come Amazon RDS) senza alcuna modifica al codice.

Allo stesso modo, un server SMTP locale dovrebbe poter essere cambiato con uno third party (come Postmark). In entrambi i casi, a cambiare dovrebbero essere solo i file di configurazione necessari.

Ogni backing service è quindi definibile come una “risorsa connessa”. Un Database MySQL è una risorsa. Due database MySQL (utilizzati per lo sharding a livello di applicazione) saranno visti come due distinte risorse. Un’app twelve-factor vede questi database come risorse anche per sottolineare la separazione dal deployment a cui fanno riferimento.



Le risorse possono essere collegate e scollegate da un deployment a piacimento. Per esempio, supponiamo che il database dell'applicazione si comporti in modo anomalo per problemi hardware.

L'amministratore potrebbe decidere di voler configurare un altro server di database ripreso da un recente backup.

Il vecchio database verrebbe scollegato, quello nuovo connesso – senza modifiche al codice.



**Build, release,
esecuzione**

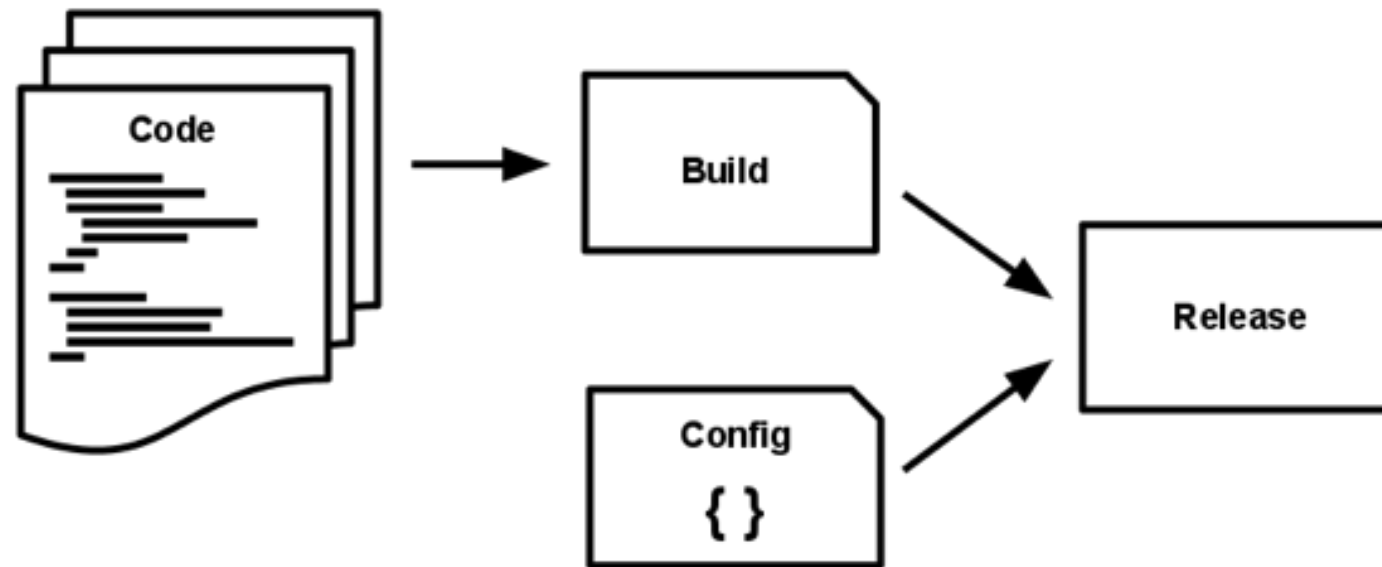
Separare in modo netto lo stadio di build dall'esecuzione.

Una codebase viene “trasformata” in deployment attraverso tre fasi:

- **la fase di build, che converte il codice del repo in una build “eseguibile”. Usando una certa versione del codice, a una specifica commit, nella fase di build vengono compilati i binari con gli asset appropriati includendo anche le eventuali dipendenze;**
- **la fase di release prende la build prodotta nella fase precedente e la combina con l'attuale insieme di impostazioni di configurazione del deployment specifico. La release risultante contiene sia la build che le impostazioni;**
- **la fase di esecuzione (conosciuta anche come “runtime”) vede l'applicazione in esecuzione nell'ambiente di destinazione, attraverso l'avvio di processi della release scelta;**

Un'app twelve-factor definisce una separazione netta tra build, release ed esecuzione.

Per esempio, è impossibile effettuare dei cambiamenti del codice a runtime, dato che non c'è modo di propagare queste modifiche all'“indietro”, verso la fase di build.



I tool di deployment offrono tipicamente dei tool di gestione delle release, in particolare alcuni dedicati a un rollback verso una release precedente.

Per esempio, Capistrano memorizza le varie release in una sotto-directory chiamata releases, in cui la release attuale non è che un symlink verso la directory della release attuale.

Il comando di rollback permette di tornare indietro a una certa release velocemente.

Ogni release dovrebbe inoltre possedere un ID univoco di rilascio, come per esempio un timestamp (es. 2011-04-06-20:32:17) o un numero incrementale (es. v100).

In un certo senso, la creazione di una release è una procedura “a senso unico” e una certa release non può essere modificata dopo la sua creazione.

Qualsiasi cambiamento deve quindi prevedere una nuova release.

Una fase di build è sempre avviata da uno sviluppatore, non appena il codice viene modificato.

Al contrario, l'esecuzione può essere anche gestita in modo automatico (si pensi al riavvio del server oppure a un crash con successivo riavvio del processo).

A ogni modo, una volta in esecuzione, la regola aurea è di evitare il più possibile (se non del tutto) modifiche che potrebbero rompere qualche equilibrio.

Magari nel bel mezzo della notte, quando non c'è nessuno disponibile.

La fase di build può essere sicuramente più “faticosa”, comunque, visto che possono verificarsi degli errori da risolvere prima di proseguire.



Processi

Esegui l'applicazione come uno o più processi stateless.

L'app viene eseguita nell'ambiente di esecuzione come uno o più processi.

Nel caso più semplice, il codice non è che uno script stand-alone, l'ambiente di esecuzione è il laptop dello sviluppatore e il processo viene lanciato tramite linea di comando (per esempio, `python my_script.py`).

Tuttavia, il deployment in produzione di un'app sofisticata potrebbe usare più tipologie di processo, istanziate in zero o più processi.

I processi twelve-factor sono stateless (senza stato) e share-nothing.

Tutti i dati che devono persistere devono essere memorizzati in un backing service, come per esempio un database.

Lo spazio di memoria o il filesystem di un processo possono essere visti come una “singola transazione” breve.

Come il download di un file, le successive operazioni su di esso e infine la memorizzazione del risultato sul database.

Un'app twelve-factor non assume mai che qualsiasi cosa messa in cache sarà poi disponibile successivamente – con tanti processi in esecuzione, le possibilità che una certa richiesta venga servita da un altro processo sono molto alte.

Comunque, anche nel caso in cui si usi un singolo processo in esecuzione, un riavvio (dovuto a deployment di codice, cambio di file di configurazione e così via) resetterà lo stato in cui si trova il sistema.

I packager di asset (come Jammit o django-compressor) usano il filesystem come cache per gli asset compilati.

Un'app twelve-factor vuole questa compilazione durante la fase di build, così come l'asset pipeline di Rails, e non a runtime.

Alcuni sistemi web si basano inoltre sulle cosiddette “sticky sessions” – che consistono nel mettere in cache i dati di sessione dell'utente presenti nella memoria del processo, aspettandosi future richieste identiche dallo stesso visitatore, venendo quindi reindirizzati allo stesso processo.

Le sticky session sono una palese violazione della metodologia twelve-factor.

I dati di sessione sono un ottimo candidato per quei sistemi di datastore che offrono la feature di scadenza, come Memcached o Redis.

Binding delle Porte

The background is a deep blue with a complex, abstract pattern. It features numerous concentric circles and radial lines that create a sense of depth and movement, similar to a tunnel or a stylized eye. The lines are lighter blue and white, contrasting with the dark blue background.

Esporta i servizi tramite binding delle porte.

Normalmente, le applicazioni web sono qualcosa di eseguito all'interno di un server web, che fa da contenitore.

Per esempio, le applicazioni PHP possono venire eseguite come modulo all'interno di Apache HTTPD, così come un'applicazione Java viene eseguita in Tomcat.

L'applicazione twelve-factor è completamente self-contained (contenuta in se stessa) e non si affida a un altro servizio (come appunto un webserver) nell'ambiente di esecuzione.

La web app esporta HTTP come un servizio effettuando un binding specifico a una porta, rimanendo in ascolto su tale porta per le richieste in entrata.

In un ambiente di sviluppo locale, lo sviluppatore accede al servizio tramite un URL come `http://localhost:5000/`.

In fase di deployment, invece, un layer di routing gestisce le richieste da un hostname pubblico alla specifica porta desiderata.

Tale funzionalità viene, frequentemente, implementata tramite dichiarazione delle opportune dipendenze, aggiungendo una libreria webserver all'applicazione come Tornado per Python, Thin per Ruby, o Jetty per Java e altri linguaggi basati su JVM.

L'evento, nella sua interezza, "ha luogo" nello spazio dell'utente, nel codice dell'applicazione.

HTTP non è l'unico servizio che può essere esportato tramite port binding. In realtà quasi ogni tipo di software può essere eseguito tramite uno specifico binding tra processo e porta dedicata.

Alcuni esempi includono ejabberd (a tal proposito, leggere su XMPP), e Redis (a proposito del protocollo Redis).

Nota inoltre che usare il binding delle porte permette a un'applicazione di diventare il backing service di un'altra applicazione, tramite un URL dedicato o comunque come una risorsa la cui configurazione si può gestire tramite appositi file di config dell'app consumer del servizio.



Concorrenza

Scalare attraverso il process model.

Ogni software, una volta avviato, è rappresentato da uno o più processi. Le web application in particolare hanno assunto nel tempo una gran varietà di forme e di tipologie di esecuzione, in tal senso.

Per esempio, i processi PHP vengono eseguiti come sotto-processi figli di Apache, avviati su richiesta quando necessari in base al volume di richieste.

Java invece gestisce le cose nella maniera opposta, tramite un superprocesso unico che usa una grande quantità di risorse sul server (CPU e memoria) dall'avvio, con una concorrenza gestita "internamente" tramite i threads.

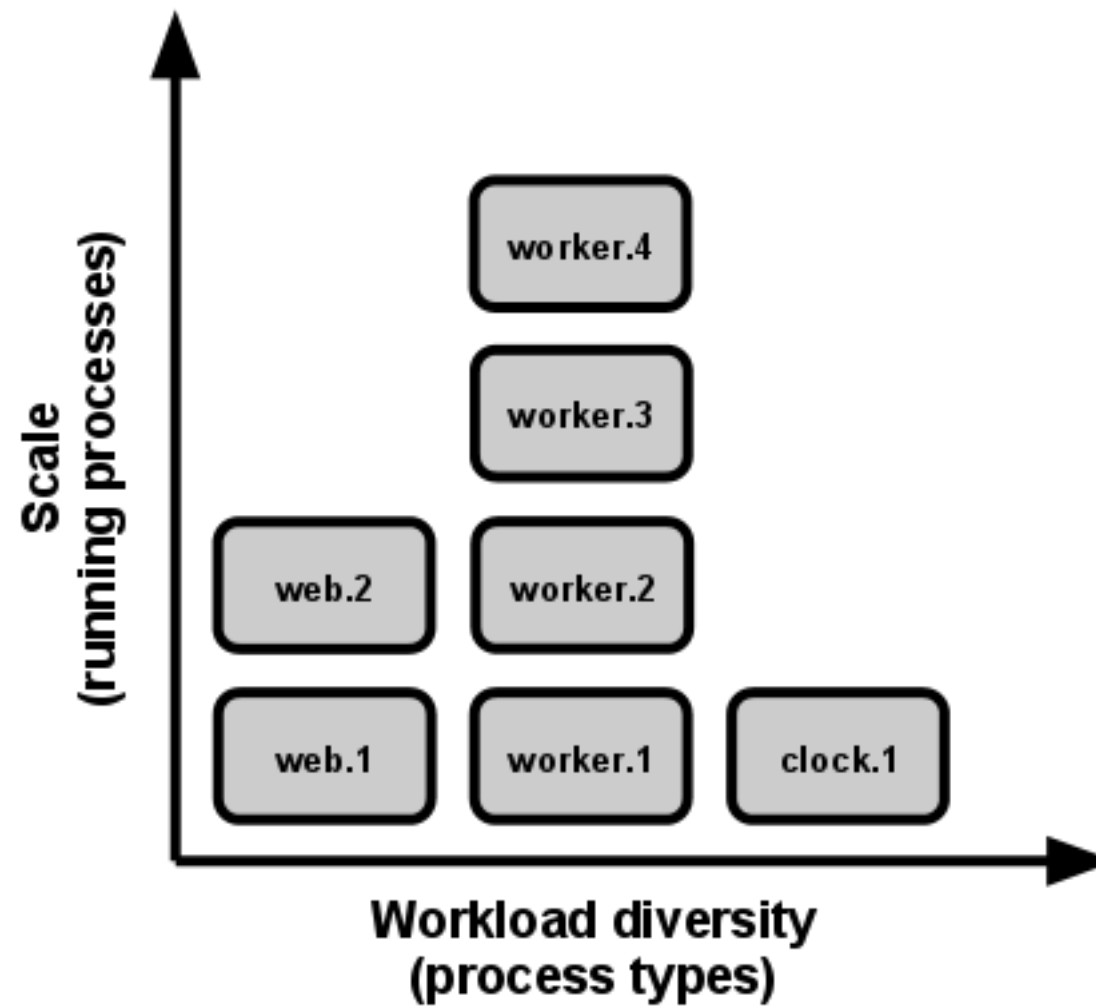
In entrambi i casi, comunque, i processi non sono esplicitamente visibili allo sviluppatore.

In un'applicazione twelve-factor, i processi sono "first class citizen".

La visione del concetto di processo prende spunto dal concetto, in unix, dedicato all'esecuzione di demoni di servizi.

Attraverso l'uso di questo modello, lo sviluppatore può realizzare la propria applicazione in modo tale da farle gestire senza problemi diversi livelli di carico di lavoro, assegnando ogni tipo di lavoro a un tipo di processo ben definito.

Per esempio, le richieste HTTP possono essere gestite da un web process, mentre dei compiti più lunghi (in background) possono essere gestiti da un altro processo separato.



Questo non esclude che un certo processo, individualmente, possa gestire in modo autonomo e interno il multiplexing, tramite threading, all'interno della VM in esecuzione, o magari un modello asincrono o basato su eventi come quello di EventMachine, Twisted, o Node.js.

Tuttavia, tutto questo può non bastare: l'applicazione deve essere anche adatta all'esecuzione su più macchine fisiche.

Il modello di processo così come presentato rende il massimo quando arriva il momento di scalare.

La natura orizzontalmente partizionabile (e non soggetta a condivisioni) di un “processo twelve-factor” permette di gestire la concorrenza in modo semplice e affidabile.

L’array dei tipi di processo e il numero di processi presenti per ogni tipo è conosciuto come process formation (formazione di processi).

I processi di un’app twelve-factor non dovrebbero essere soggetti a daemonizing, o scrivere file PID.

Al contrario, facendo affidamento sul process manager del sistema operativo (come systemd, un process manager distribuito su piattaforma cloud, o tool come Foreman durante lo sviluppo) per gestire gli stream in output, rispondere adeguatamente a crash di processi e gestire riavvii e shutdown.



Rilasciabilità

Massimizzare la robustezza con avvii veloci e shutdown graduali.

I processi di un'applicazione twelve-factor sono rilasciabili, cioè possono essere avviati o fermati senza problemi al momento del bisogno.

Questa caratteristica ovviamente facilita le procedure di scaling, deploy rapido della codebase o cambi dei file di configurazione.

I processi dovrebbero inoltre ambire a minimizzare i tempi di avvio. Idealmente, un processo impiega pochi secondi dal tempo di lancio al momento in cui tutto è pronto per ricevere nuove richieste.

Dei tempi brevi di avvio inoltre forniscono una maggiore agilità in fase di release; il tutto a vantaggio della robustezza dell'applicazione, dato che il process manager può così muoversi agevolmente verso un'altra macchina fisica, se necessario.

I processi dovrebbero inoltre terminare in modo tutt'altro che brusco, quindi graduale, in caso di ricezione di un segnale SIGTERM dal process manager.

Per un'applicazione web, la giusta terminazione di un processo viene ottenuta quando si cessa innanzitutto di ascoltare sulla porta dedicata del servizio (evitando quindi di ricevere altre richieste), permettendo poi di terminare le richieste esistenti e infine di concludere la fase di terminazione in modo definitivo.

Per un processo worker, invece, la fase di terminazione più adatta vede il ritorno del job corrente alla coda.

Per esempio, su RabbitMQ il worker può inviare un NACK;

Su Beanstalkd, il job viene automaticamente rimandato in coda nel caso in cui il worker si disconnette.

I sistemi basati su lock come [Delayed]

I processi dovrebbero, inoltre, essere “robusti nei confronti di situazioni di crash improvviso”, cosa che si verifica ad esempio in caso di problemi a livello di hardware sottostante.

Nonostante questa seconda evenienza si verifichi meno frequentemente di una chiusura con SIGTERM, può comunque succedere.

L’approccio raccomandato, in questi casi, è l’uso di un sistema robusto di code, come Beanstalkd, che rimanda il job in coda in caso di timeout o disconnessione.

Ad ogni modo, una buona applicazione twelve-factor deve poter gestire senza problemi le terminazioni inaspettate. Il Crash-only design porta questo concetto alla sua logica conclusione.

The background is a dark blue field filled with a complex pattern of concentric circles and radial lines, creating a tunnel-like or orbital effect. The lines are lighter blue and have a glowing, slightly grainy texture, suggesting a digital or scientific theme.

Parità tra Sviluppo e Produzione

Mantieni lo sviluppo, staging e produzione simili il più possibile.

Storicamente, ci sono sempre state differenze sostanziali tra gli ambienti di sviluppo (lo sviluppatore che effettua delle modifiche live a un deployment in locale) e quello di produzione (un deployment in esecuzione raggiungibile dagli utenti finali).

Differenze (o gap) che si possono raggruppare in tre categorie:

- **Tempo:** uno sviluppatore può lavorare sul codice per giorni, settimane o mesi prima di poter andare in produzione;
- **Personale:** gli sviluppatori scrivono il codice, gli ops effettuano il deployment;
- **Strumenti:** gli sviluppatori potrebbero usare uno stack quale Nginx, SQLite e OS X, mentre in produzione per il deployment verrebbero installati Apache, MySQL e Linux.

Un'applicazione twelve-factor è progettata per il rilascio continuo, tenendo così queste differenze al minimo possibile.

A proposito di queste tre tipologie di differenze appena viste:

Rendi la differenze temporali minime: cerca di scrivere (o far scrivere) del codice da rilasciare nel giro di poche ore, se non minuti;

Rendi le differenze a livello di personale minime: fai in modo che gli sviluppatori siano coinvolti anche nella fase di deploy, per permettere loro di osservare il comportamento di ciò che hanno scritto anche in produzione;

Rendi le differenze a livello di strumenti minime: mantieni gli ambienti di lavoro il più simile possibile;

Riassumendo tutto in una tabella:

	App Tradizionale	App Twelve-factor
Tempo tra i Deployment	Settimane	Ore
Sviluppatori e Ops	Sono diversi	Sono gli stessi
Sviluppo e Produzione	Divergenti	Il più simili possibile

I backing service, come il database dell'applicazione o la cache, sono una delle aree in cui la parità degli ambienti è molto importante. Molti linguaggi offrono delle librerie che facilitano l'accesso a questi servizi, tra cui anche degli adattatori per questi tipi di servizi. Eccone alcuni:

Tipologia	Linguaggio	Libreria	Adattatore
Database	Ruby/Rails	ActiveRecord	MySQL, PostgreSQL, SQLite
Code	Python/Django	Celery	RabbitMQ, Beanstalkd, Redis
Cache	Ruby/Rails	ActiveSupport::Cache	Memory, filesystem, Memcached

Gli sviluppatori, inoltre, trovano utile usare dei servizi “leggeri” in fase di sviluppo, passando quindi a qualcosa di più serio e robusto in produzione.

Per esempio, usando SQLite localmente e PostgreSQL in produzione.

Ancora, un sistema di cache in locale in fase di sviluppo e Memcached in produzione.

Lo sviluppatore twelve-factor “resiste” a questa necessità, anche se gli adapter ci sono e funzionano in modo tale da astrarre in modo sufficiente tutte le differenze nella gestione.

Nulla impedisce, infatti, a qualche altra incompatibilità di uscire allo scoperto quando meno ce lo si aspetta, soprattutto se in ambiente di sviluppo funziona tutto e poi, magari, in produzione i test non vengono superati.

Il costo di questa differenza può risultare abbastanza alto, soprattutto in situazioni in cui si effettua il rilascio continuo.

Rispetto al passato, usare dei sistemi “light” in locale è una prassi poco convincente.

Si pensi al fatto che alcuni servizi moderni come Memcached o PostgreSQL si possono installare e usare senza difficoltà tramite alcuni sistemi di packaging come Homebrew e apt-get.

In alternativa, esistono anche alcuni tool di provisioning come Chef e Puppet, che combinati con sistemi di ambienti virtuali come Vagrant permettono agli sviluppatori di riprodurre in locale delle macchine molto simili, se non identiche, a quelle in produzione.

Ne risente quindi positivamente il costo di deployment.

Tutto questo, sia chiaro, non rende gli adapter meno utili: grazie ad essi infatti il porting verso nuovi servizi, in un secondo momento, rimane un processo indolore.

Nonostante questo, comunque, rimane scontato che sarebbe buona norma usare uno stesso backing service su tutti i deployment di un'applicazione.



LOG

Tratta i log come stream di eventi.

I log offrono una maggiore chiarezza riguardo un comportamento di un'app in esecuzione.

In ambienti basati su server, questi sono tendenzialmente scritti su un file su disco (logfile).

A ogni modo, è solo un formato.

Un log può essere definito infatti come uno stream di eventi aggregati e ordinati cronologicamente.

Tali eventi vengono presi da tutti i vari output stream presenti di tutti i processi attivi, oltre che dai vari backing service.

Nella loro forma grezza, i log si presentano come un file di testo con un evento per ogni linea (con le dovute eccezioni).

Non hanno un inizio o una fine ben definiti, ma un continuo di informazioni fin quando l'applicazione è al lavoro.

Un'applicazione twelve-factor non dovrebbe preoccuparsi di lavorare con il proprio output stream.

Non dovrebbe lavorare o comunque gestire i vari logfile.

Dovrebbe, invece, fare in modo che ogni processo si occupi di scrivere il proprio stream di eventi su “stdout”.

Durante lo sviluppo in locale, quindi, lo sviluppatore potrà visionare lo stream in modo completo direttamente dal terminale, per capire meglio il comportamento della sua applicazione.

In staging o in produzione, invece, ogni stream viene “preso” dall’ambiente di esecuzione ed elaborato insieme a tutti gli altri stream dell’applicazione, quindi indirizzato verso una o più “destinazioni” finali per la visualizzazione e archiviazione a lungo termine.

Queste “destinazioni” non sono visibili o configurabili, ma vengono gestite totalmente dall’ambiente di esecuzione.

Per questi scopi esistono strumenti come Logplex e Fluentd).

Uno stream di eventi di un'applicazione può essere quindi indirizzato verso un file, o visionato in tempo reale su un terminale.

Ancora, lo stream può essere inviato a un sistema di analisi e indicizzazione di log, come Splunk, oppure a un sistema di memorizzazione general-purpose come Hadoop/Hive.

Questi sistemi hanno ottimi tool per effettuare un lavoro di analisi del comportamento dell'applicazione, come per esempio:

- **Ricerca di specifici eventi nel passato;**
- **Grafici per rappresentare dei trend (es. richieste per minuto);**
- **Attivazione di alert specifici in base a regole definite dall'utente (es. alert avverte l'amministratore se il rate di eventi al minuto sale oltre una certa soglia);**

The background is a dark blue field filled with a complex pattern of concentric circles and radial lines, creating a tunnel-like or orbital effect. The lines are lighter blue and have a glowing, slightly grainy texture. The overall composition is centered and symmetrical.

Processi di Amministrazione

Esegui i task di amministrazione come processi una tantum.

La “process formation” è l’array dei processi che vengono usati durante le normali operazioni dell’applicazione (per esempio, la gestione delle richieste web). Non è tutto, però: ci sono dei task che lo sviluppatore può voler eseguire, una volta ogni tanto. Per esempio:

- **Esecuzione delle migration del database (esempi: `manage.py migrate` in Django, `rake db:migrate` in Rails).**
- **Esecuzione di una console (una REPL shell) in modo tale da avviare del codice arbitrariamente o analizzare alcuni aspetti dell’applicazione specifici. Molti linguaggi prevedono una REPL, in genere avviando l’interprete senza opzioni e argomenti aggiuntivi (esempi: `python` o `perl`) o in alcuni casi eseguibile con un comando separato (esempi: `irb` per Ruby, `rails console` per Rails).**
- **Esecuzione one-time di alcuni script specifici (esempio: `php scripts/fix_bad_records.php`).**

Tali processi dovrebbero essere avviati in un ambiente identico a quello in cui lavorano gli altri nel contesto dell'applicazione.

Dovrebbero essere eseguiti quindi su una specifica release, partendo dalla stessa codebase e impostazioni di configurazione.

Il codice per l'amministrazione dovrebbe inoltre essere incluso nel codice dell'applicazione, in modo tale da evitare qualsiasi problema di sincronizzazione.

La stessa tecnica di isolamento delle dipendenze dovrebbe poter essere usata allo stesso modo su tutti i processi.

Per esempio, se il processo web di Ruby può usare il comando `bundle exec thin start`, una migration del database dovrebbe poter usare `bundle exec rake db:migrate` senza problemi.

Allo stesso modo, un programma Python che usa Virtualenv dovrebbe usare il `bin/python` per eseguire sia i server Tornado che processi di amministrazione.

La metodologia twelve-factor favorisce molto tutti quei linguaggi che offrono una shell REPL out of the box, rendendo quindi semplice l'esecuzione di script una tantum.

In un deployment locale, gli sviluppatori possono invocare questi processi speciali tramite un semplice comando diretto.

In un ambiente di produzione, invece, gli sviluppatori possono raggiungere lo stesso obiettivo tramite SSH o un qualsiasi altro sistema di esecuzione di comandi remoto.